

Highlights

Neighbourhood Structures and Ranking Operators for the Interval Job Shop Scheduling Problem

Hernán Díaz Rodríguez

- Unified extreme critical path framework for neighbourhoods in the interval JSP.
- Feasibility, connectivity and no-loss proofs for any admissible interval ranking.
- LEX2 yields 5–11% narrower intervals by tightening the worst-case critical path.
- N_{ext} is the strongest neighbourhood after tuning; N_3 worst on quality and runtime.
- N_8 competitive on CRISP; midpoint filter counter-productive on intervals.

Neighbourhood Structures and Ranking Operators for the Interval Job Shop Scheduling Problem

Hernán Díaz Rodríguez^{a,*}

^a*Department of Computing, University of Oviedo, Campus de Gijón, Gijón, 33204, Spain*

Abstract

The Job Shop Scheduling Problem with interval-valued processing times (IJSP) models real-world uncertainty where durations are unknown but bounded. Comparing schedules then requires an interval ranking operator, and the classical notion of critical path must be extended to both extreme scenarios. I formalise *extreme critical paths* — paths critical in the best-case or worst-case scenario graph — and use them to define a neighbourhood $H(\sigma)$ for local search. I prove that $H(\sigma)$ is feasible, connected and loses no improving neighbours under any admissible interval ranking. Five neighbourhood variants (N_1 , N_2 , N_3 , N_{ext} , N_8) are formulated within this framework and combined with four ranking operators (EV, LEX1, LEX2, YX) on 82 benchmark instances of up to 1000 operations. Two findings are noteworthy: (i) after irace-based tuning, N_{ext} is the strongest neighbourhood, followed by N_1 and N_2 (negligible to small effects), with N_8 and N_3 trailing by a medium effect; and (ii) LEX2 yields makespan intervals 5–11% narrower than the alternatives on average. A float analysis of both extreme graphs is consistent with this finding: LEX2 solutions show more critical operations and less per-operation slack in the worst-case graph, supporting the interpretation that LEX2 directly tightens the worst-case critical path rather than leaving structural slack. A validation on the 82 deterministic counterparts (CRISP) confirms that the relative ranking of neighbourhoods carries over to the classical setting, and that N_8 becomes competitive when uncertainty disappears.

Keywords: Job shop scheduling, combinatorial optimisation, local search, neighbourhood structures, interval uncertainty, interval ranking operators

*Corresponding author

Email address: diazhernan@uniovi.es (Hernán Díaz Rodríguez)

1. Introduction

The Job Shop Scheduling Problem (JSP) is a flagship combinatorial optimisation problem in operations research, and neighbourhood-based local search has been its most competitive algorithmic paradigm for three decades [1, 2, 3]. The effectiveness of these methods rests almost entirely on neighbourhood design: the critical-block insight of [2] — that reversing interior arcs of a critical block cannot improve the makespan — reduces neighbourhood size by two-thirds without sacrificing solution quality, and the extended neighbourhood of [3] refines this further with a heads-and-tails viability filter. In practice, however, processing times are rarely known precisely at planning time: operator variability, machine condition, and material heterogeneity introduce uncertainty that deterministic models cannot capture.

A natural and tractable representation for such uncertainty is the *closed interval* $[p^-, p^+]$, bounding the minimum and maximum possible durations [4, 5]. The resulting *Interval Job Shop Scheduling Problem (IJSP)* raises two intertwined challenges: comparing schedules requires an *interval ranking operator* [6, 7], since the makespan is now an interval; and the critical-path concept that underpins efficient neighbourhood design must be reconsidered, because different paths may become critical under different realisations of the processing times [8, 9, 10].

1.1. Contributions

This paper makes the following contributions.

1. Formal definition of extreme criticality for IJSP (Section 3). I define a task or arc as *extreme critical* if it lies on a critical path of the best-case graph $G^-(\sigma)$ or the worst-case graph $G^+(\sigma)$, and prove that this definition encompasses all necessarily critical tasks and is contained within the set of possibly critical tasks, making it a computationally tractable and theoretically sound approximation of the general criticality concept of [9].
2. Neighbourhood $H(\sigma)$ based on extreme critical arcs (Section 4). I define $H(\sigma)$ as the set of all feasible processing orders reachable by reversing a single extreme critical disjunctive arc, and prove three structural properties: feasibility (every move yields a feasible order), no loss of improving neighbours (moves outside $H(\sigma)$ cannot improve the makespan

- under any admissible ranking), and connectivity (every non-optimal order can reach a global optimum via a finite sequence of moves in $H(\sigma)$).
3. Five neighbourhood variants within the $H(\sigma)$ framework (Section 5). I formulate N_1 , N_2 , N_3 , N_{ext} and a new variant N_8 (boundary arc swaps extended with extra-block reinsertion moves, after [11]) as instances of $H(\sigma)$, characterising their theoretical size and the cost of generating them.
 4. Empirical study on 82 benchmark instances (Section 6). Three experimental phases are reported: (a) a 5×4 comparison of five neighbourhoods and four operators at a common parameter setting (Section 6.2); (b) a head-to-head comparison of the five neighbourhoods after irace-based hyperparameter tuning, isolating the structural effect of the neighbourhood (Section 6.5); and (c) a validation phase on the 82 deterministic counterparts of the same instances (CRISP, Section 6.6).
 5. Discovery and structural explanation of the LEX2-width effect (Sections 6.3 and 6.4). LEX2 produces solutions with $\sim 5\text{--}11\%$ narrower makespan intervals than the other operators on average. A float analysis of both extreme graphs falsifies the natural “more slack” hypothesis and provides structural evidence for this effect: LEX2 solutions show more critical operations and less slack in $G^+(\sigma)$, consistent with LEX2 tightening the worst-case critical path directly.

1.2. Paper Organisation

Section 2 introduces the IJSP formally and reviews related work on critical paths and rankings. Section 3 defines extreme criticality. Section 4 defines the neighbourhood $H(\sigma)$ and proves its three structural properties. Section 5 describes the five neighbourhood variants. Section 6 reports the experimental study. Section 7 discusses the findings, and Section 8 concludes.

2. Background and Related Work

2.1. The Deterministic Job Shop Problem

The JSP consists of n jobs $\{J_1, \dots, J_n\}$ and m machines $\{M_1, \dots, M_m\}$. Each job J_i is a sequence of operations $O_{i,1}, \dots, O_{i,m}$, where operation $O_{i,j}$ must be processed on machine M_j for exactly $p_{i,j}$ time units. Operations within a job must respect their precedence order, and each machine processes at most one operation at a time. The goal is to find a *feasible processing order* σ minimising the makespan $C_{\max}$.

The *solution graph* $G(\sigma) = (V, A \cup D(\sigma))$ represents a feasible order: V is the set of operations augmented with a dummy source node S and sink node T ; A contains precedence arcs within jobs; and $D(\sigma)$ contains disjunctive arcs imposed by the order σ on each machine. The makespan equals the length of the longest path in $G(\sigma)$. The classical insight of [1] is that the neighbourhood for local search should restrict to reversals of arcs on critical paths, ensuring feasibility, while [2] shows that reversals of *interior* arcs of a critical block cannot improve the makespan, allowing further pruning.

2.2. Interval Arithmetic and Ranking Operators

When processing times are uncertain, I represent each as $\mathbf{p} = [p^-, p^+] \in \mathbb{IR}$, where $\mathbb{IR}$ denotes the set of closed intervals. The IJSP requires three arithmetic operations on $\mathbb{IR}$ [4]:

$$\mathbf{a} + \mathbf{b} = [a^- + b^-, a^+ + b^+], \quad (1)$$

$$\max(\mathbf{a}, \mathbf{b}) = [\max(a^-, b^-), \max(a^+, b^+)]. \quad (2)$$

Since there is no natural total order on $\mathbb{IR}$, comparing schedules requires choosing a *ranking operator*. I consider four rankings widely used in the literature [6, 12, 5]:

$$\mathbf{a} \leq_{\text{LEX1}} \mathbf{b} \iff a^- < b^- \vee (a^- = b^- \wedge a^+ \leq b^+) \quad (3)$$

$$\mathbf{a} \leq_{\text{LEX2}} \mathbf{b} \iff a^+ < b^+ \vee (a^+ = b^+ \wedge a^- \leq b^-) \quad (4)$$

$$\mathbf{a} \leq_{\text{YX}} \mathbf{b} \iff (a^- + a^+) < (b^- + b^+) \vee ((a^- + a^+) = (b^- + b^+) \wedge (a^+ - a^-) \leq (b^+ - b^-)) \quad (5)$$

$$\mathbf{a} \leq_{\text{EV}} \mathbf{b} \iff \frac{a^- + a^+}{2} \leq \frac{b^- + b^+}{2} \quad (6)$$

LEX1, LEX2 and YX are *admissible* total orders on $\mathbb{IR}$ [6]: they refine the partial product order $\leq_2$ defined by $\mathbf{a} \leq_2 \mathbf{b} \iff a^- \leq b^- \wedge a^+ \leq b^+$. EV ranks intervals by their midpoint $\frac{a^- + a^+}{2}$ and induces a total preorder rather than a strict total order, since two distinct intervals may share the same midpoint; it also refines $\leq_2$. This refinement property is essential for the theoretical guarantees of Section 4: any move that simultaneously decreases both $C_{\max}^-$ and $C_{\max}^+$ is an improvement under all four rankings considered here.

2.3. Related Work on IJSP Neighbourhoods

The first neighbourhood for the fuzzy/interval JSP was proposed in [10], extending the van Laarhoven neighbourhood [1] to use possibly critical arcs. This corresponds to what I call N_1 in the framework. Subsequent work introduced N_2 as a filtered version of N_1 that retains only block-boundary arcs [13], reducing neighbourhood size by $\sim 64\%$ with statistically identical solution quality. N_3 extends the neighbourhood to consider subsequence reversals [14]. For the related fuzzy job shop, evolutionary tabu search combining these neighbourhood structures with genetic operators has been proposed for due-date satisfaction objectives [15]. Prior approaches to the IJSP specifically have combined neighbourhood search with genetic algorithms for makespan [16] and tardiness minimisation [17], providing early evidence that the choice of ranking operator influences solution quality.

The Nowicki–Smutnicki extended neighbourhood [3], which I adapt here under the name N_{ext} , was originally proposed for the crisp JSP and provides strong empirical performance. Its adaptation to the IJSP, presented in this work, takes advantage of the fact that the boundary-arc result of [2] can be lifted simultaneously to both extreme graphs $G^-(\sigma)$ and $G^+(\sigma)$. I also propose a fifth variant, N_8 , which extends N_2 with extra-block reinsertion moves that relocate critical block endpoints to positions outside the block on the same machine [11]. For the deterministic JSP and FJSP, a general framework unifying feasibility checking and quality evaluation of moves for any regular criterion was proposed in [18]; tight lower and upper bounds that accelerate this evaluation without full re-computation were subsequently developed in [19]. Analogous efficiency improvements for the interval setting remain an open direction. In a complementary line of work, machine learning methods have been proposed to estimate quantile-based risk measures (VaR, CVaR) of the makespan in interval JSP [20].

3. Extreme Criticality for IJSP

3.1. Configurations and Extreme Scenario Graphs

A *configuration* $\omega = (p_1, \dots, p_o)$, where $o = n \times m$, assigns a deterministic duration $p_i \in \mathbf{p}_i$ to each operation i ; let Ω denote the set of all configurations. Given a feasible processing order σ , two *extreme scenario graphs* are of particular interest:

- $G^-(\sigma)$: identical to $G(\sigma)$ but with each arc (x, y) weighted by p_x^- (best case).

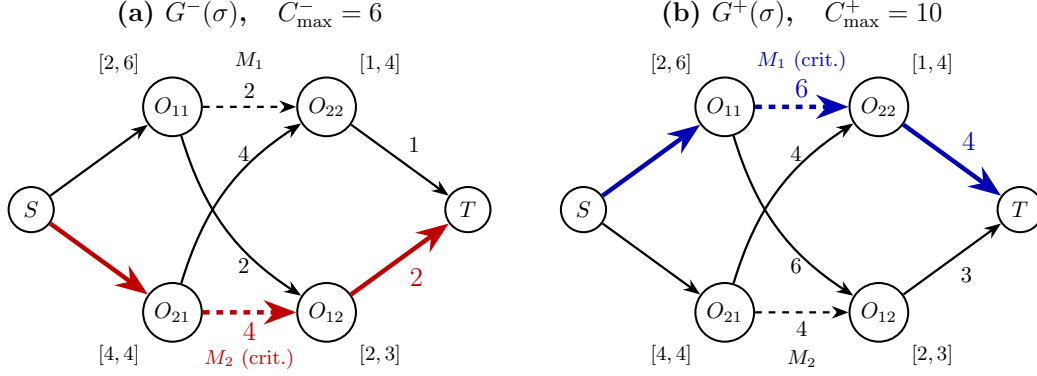


Figure 1: Illustrative example. (a) Best-case graph $G^-(\sigma)$ with its critical path highlighted in red. (b) Worst-case graph $G^+(\sigma)$ with its (different) critical path highlighted in blue. Precedence arcs are drawn solid, disjunctive arcs dashed.

- $G^+(\sigma)$: identical to $G(\sigma)$ but with each arc (x, y) weighted by p_x^+ (worst case).

These correspond to the extreme configurations $\omega^+(\emptyset)$ and $\omega^+(V)$ of [9]. The makespan interval satisfies

$$\mathbf{C}_{\max}(\sigma) = [C_{\max}^-(\sigma), C_{\max}^+(\sigma)], \quad (7)$$

where $C_{\max}^-(\sigma)$ is the length of the longest path in $G^-(\sigma)$ and $C_{\max}^+(\sigma)$ is the length of the longest path in $G^+(\sigma)$. An illustrative example is shown in Figure 1: two jobs ($J_1: O_{11}(M_1, [2, 6]) \rightarrow O_{12}(M_2, [2, 3])$; $J_2: O_{21}(M_2, [4, 4]) \rightarrow O_{22}(M_1, [1, 4])$), with schedule σ fixing M_1 order (O_{11}, O_{22}) and M_2 order (O_{21}, O_{12}). In $G^-(\sigma)$ the critical path runs through the M_2 disjunctive arc ($C_{\max}^- = 6$); in $G^+(\sigma)$ it runs through the M_1 disjunctive arc ($C_{\max}^+ = 10$). The critical arcs in the two extreme graphs differ, motivating the need to consider both.

3.2. Heads, Tails and Floats

For each extreme graph $G^*(\sigma)$ ($* \in \{-, +\}$), the *head* r_x^* of task x is the length of the longest path from the dummy start node to x , and the *tail* q_x^* is the length of the longest path from the completion of x to the end node. The *float* of x in $G^*(\sigma)$ is

$$F_x^*(\sigma) = C_{\max}^*(\sigma) - r_x^* - p_x^* - q_x^*. \quad (8)$$

A task is critical in $G^*(\sigma)$ iff $F_x^*(\sigma) = 0$. The general problem of computing the minimum float $\min_{\omega \in \Omega} F_x(\sigma, \omega)$ over all configurations — which equals zero iff x is possibly critical — is NP-hard [21, 22]; efficient branch-and-bound algorithms for this harder problem are given in [23]. The extreme-scenario floats F_x^- and F_x^+ used here are computable in linear time and provide the tractable approximation exploited in Sections 4 and 5.

3.3. Definition and Connection to Possibly/Necessarily Critical Tasks

Definition 1 (Extreme critical path). *A path P in $G(\sigma)$ is extreme critical if it is critical in $G^-(\sigma)$ or in $G^+(\sigma)$. A node, arc or block is extreme critical if it lies on an extreme critical path.*

This definition connects to the general criticality concepts of [21, 9] as follows. A task x is *possibly critical* under σ if there exists $\omega \in \Omega$ such that $F_x(\sigma, \omega) = 0$; it is *necessarily critical* if $F_x(\sigma, \omega) = 0$ for every $\omega \in \Omega$ [21, 9].

Proposition 1. *Every extreme critical task is possibly critical. Every necessarily critical task is extreme critical.*

Proof. (i) *Extreme critical $\Rightarrow$ possibly critical.* Let task x be extreme critical; then $F_x^-(\sigma) = 0$ or $F_x^+(\sigma) = 0$. The extreme configurations $\omega^- = (p_i^-)_i$ and $\omega^+ = (p_i^+)_i$ both belong to Ω (since $p_i^\pm \in [p_i^-, p_i^+]$ by definition). In case $F_x^-(\sigma) = 0$: the float of x under configuration ω^- equals $F_x(\sigma, \omega^-) = C_{\max}^-(\sigma) - r_x^- - p_x^- - q_x^- = F_x^-(\sigma) = 0$, so x lies on a critical path for $\omega^- \in \Omega$, making it possibly critical [21, 9]. The case $F_x^+(\sigma) = 0$ is analogous with ω^+ .

(ii) *Necessarily critical $\Rightarrow$ extreme critical.* If x is necessarily critical, then $F_x(\sigma, \omega) = 0$ for every $\omega \in \Omega$. Taking $\omega = \omega^- \in \Omega$ gives $F_x^-(\sigma) = 0$, so x is critical in $G^-(\sigma)$, hence extreme critical. $\square$

The practical consequence is that extreme criticality offers a computationally tractable approximation to the exact criticality predicates: computing critical paths in $G^-(\sigma)$ and $G^+(\sigma)$ each costs $O(|A \cup D(\sigma)|)$ via standard longest-path algorithms, while computing the full set of possibly critical tasks is NP-hard in general [21, 9].

4. The Neighbourhood $H(\sigma)$ and Its Properties

4.1. Definition

Definition 2 (Neighbourhood $H(\sigma)$). *Let $\sigma_{(v)}$ denote the processing order obtained from σ by reversing a single disjunctive arc $v \in D(\sigma)$. The neigh-*

bourhood is

$$H(\sigma) = \{\sigma_{(v)} : v \in D(\sigma) \text{ is extreme critical}\}. \quad (9)$$

For comparison, the corresponding neighbourhood based on possibly critical arcs is

$$H'(\sigma) = \{\sigma_{(v)} : v \in D(\sigma) \text{ is possibly critical}\}. \quad (10)$$

By Proposition 1, $H(\sigma) \subseteq H'(\sigma)$, with strict inclusion in general.

4.2. No Loss of Improving Neighbours

Proposition 2. *Let $\sigma \in \Sigma$ be a feasible order and let $v \in D(\sigma)$ be a disjunctive arc that is not extreme critical in $G(\sigma)$. Then*

$$C_{\max}^-(\sigma) \leq C_{\max}^-(\sigma_{(v)}) \quad \text{and} \quad C_{\max}^+(\sigma) \leq C_{\max}^+(\sigma_{(v)}), \quad (11)$$

and consequently $\mathbf{C}_{\max}(\sigma) \leq_R \mathbf{C}_{\max}(\sigma_{(v)})$ for every ranking $R \in \{\text{EV}, \text{LEX1}, \text{LEX2}, \text{YX}\}$.

Proof. We prove $C_{\max}^*(\sigma) \leq C_{\max}^*(\sigma_{(v)})$ for $* \in \{-, +\}$; both cases are identical.

Since (x, y) is not extreme critical, it lies on no critical path of $G^*(\sigma)$. Let P^* be any critical path of $G^*(\sigma)$, of length $C_{\max}^*(\sigma)$. P^* does not contain arc (x, y) . Since $G^*(\sigma_{(v)})$ differs from $G^*(\sigma)$ only by replacing (x, y) with (y, x) , every arc of P^* is still present in $G^*(\sigma_{(v)})$. Hence P^* is a valid directed path in $G^*(\sigma_{(v)})$ of weight $C_{\max}^*(\sigma)$, so $C_{\max}^*(\sigma_{(v)}) \geq C_{\max}^*(\sigma)$.

Applied to both extreme graphs: $C_{\max}^-(\sigma_{(v)}) \geq C_{\max}^-(\sigma)$ and $C_{\max}^+(\sigma_{(v)}) \geq C_{\max}^+(\sigma)$, i.e., $\mathbf{C}_{\max}(\sigma) \leq_2 \mathbf{C}_{\max}(\sigma_{(v)})$. Since every ranking $R \in \{\text{EV}, \text{LEX1}, \text{LEX2}, \text{YX}\}$ refines $\leq_2$, we obtain $\mathbf{C}_{\max}(\sigma) \leq_R \mathbf{C}_{\max}(\sigma_{(v)})$. $\square$

Corollary 1. *Neighbours in $H'(\sigma) \setminus H(\sigma)$ can never improve the makespan under any of the four rankings considered.*

Corollary 1 justifies using $H(\sigma)$ instead of the larger $H'(\sigma)$: the reduction is gained at zero cost in solution quality.

4.3. Feasibility

Theorem 1 (Feasibility). *For every feasible $\sigma \in \Sigma$, the reversal of any extreme critical arc $v = (x, y) \in D(\sigma)$ produces a feasible processing order $\sigma_{(v)} \in \Sigma$. Hence $H(\sigma) \subseteq \Sigma$.*

Proof. Suppose for contradiction that $G(\sigma_{(v)})$ contains a directed cycle. Since $G(\sigma)$ is acyclic and $G(\sigma_{(v)}) = (G(\sigma) \setminus \{(x, y)\}) \cup \{(y, x)\}$, any cycle in $G(\sigma_{(v)})$ must traverse the newly added arc (y, x) . Therefore there exists a directed path π from x to y in $G(\sigma_{(v)})$ that avoids (y, x) . All arcs of π belong to $G(\sigma) \setminus \{(x, y)\}$ (the arc (x, y) was removed, and π uses none of the new arcs); so π is a valid path from x to y in $G(\sigma)$ not using (x, y) .

In the standard JSP, x and y belong to the same machine but to different jobs; hence no job-precedence arc of A runs directly from x to y . Therefore π has at least one intermediate operation u distinct from x and y . The length of π in $G^*(\sigma)$ is

$$\ell(\pi) = p_x^* + p_u^* + \cdots \geq p_x^* + p_u^* > p_x^*,$$

since all processing times satisfy $p_z^* \geq p_z^- > 0$.

On the other hand, (x, y) lies on a critical path of $G^*(\sigma)$ (denote it $CP^*(\sigma)$), which implies that the head of y is tight through the arc (x, y) :

$$r_y^* = r_x^* + p_x^*.$$

But the path π from x to y in $G(\sigma)$ yields

$$r_y^* \geq r_x^* + \ell(\pi) > r_x^* + p_x^* = r_y^*,$$

a contradiction. Hence $G(\sigma_{(v)})$ is acyclic, and $\sigma_{(v)} \in \Sigma$. $\square$

4.4. Connectivity

Theorem 2 (Connectivity). *For every non-optimal $\sigma \in \Sigma$, there exists a finite sequence of moves in $H(\sigma)$ leading to a globally optimal order σ^* .*

Proof. Fix any globally optimal order $\sigma^* \in \Sigma^*$ that minimises the *inversion count*

$$M(\sigma, \sigma^*) = |\{(x, y) \in D(\sigma) : (y, x) \in D(\sigma^*)\}|,$$

which counts direct machine-order disagreements between σ and σ^* . Note that $M(\sigma, \sigma^*) = 0$ if and only if $D(\sigma) = D(\sigma^*)$, i.e., $\sigma = \sigma^*$.

Key lemma. If λ is non-optimal, then there exists an extreme critical arc $(x, y) \in D(\lambda)$ with $(y, x) \in D(\sigma^*)$.

Proof. Since λ is non-optimal and every ranking R refines $\leq_2$, we have $\mathbf{C}_{\max}(\lambda) \not\leq_2 \mathbf{C}_{\max}(\sigma^*)$ (otherwise $\mathbf{C}_{\max}(\lambda) \leq_R \mathbf{C}_{\max}(\sigma^*)$, contradicting non-optimality). Thus either $C_{\max}^-(\lambda) > C_{\max}^-(\sigma^*)$ or $C_{\max}^+(\lambda) > C_{\max}^+(\sigma^*)$; assume without loss of generality the former. Let P be a critical path in $G^-(\lambda)$, of length $C_{\max}^-(\lambda)$. If every disjunctive arc (u, v) of P satisfied $(u, v) \in D(\sigma^*)$, then P would be a valid directed path in $G^-(\sigma^*)$ of length $C_{\max}^-(\lambda) > C_{\max}^-(\sigma^*)$, contradicting the definition of $C_{\max}^-(\sigma^*)$. Hence P contains a disjunctive arc $(x, y) \in D(\lambda) \setminus D(\sigma^*)$. Since for every machine pair exactly one of $(x, y), (y, x)$ belongs to $D(\sigma^*)$, we get $(y, x) \in D(\sigma^*)$. Moreover (x, y) is extreme critical because it lies on the critical path P of $G^-(\lambda)$.

□_{lemma}

Construction and termination. Build the sequence $\lambda_0 = \sigma$, and at each step k :

- (a) If λ_k is globally optimal, stop: a globally optimal order has been reached.
- (b) Otherwise, apply the key lemma to obtain an extreme critical $(x, y) \in D(\lambda_k)$ with $(y, x) \in D(\sigma^*)$. Set $\lambda_{k+1} = (\lambda_k)_{(x, y)} \in H(\lambda_k)$ (feasible by Theorem 1).

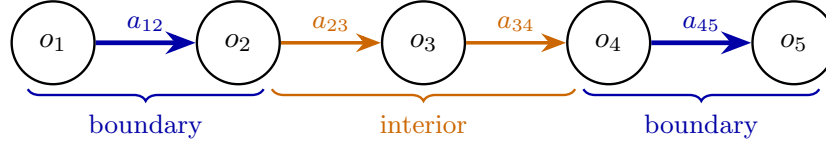
After step (b), the inversion count satisfies $M(\lambda_{k+1}, \sigma^*) = M(\lambda_k, \sigma^*) - 1$: the reversal removes (x, y) from $D(\lambda_{k+1})$ (eliminating one inversion, since $(y, x) \in D(\sigma^*)$), adds (y, x) (which agrees with σ^* , adding no inversion), and leaves all other arcs unchanged.

Since $M \geq 0$ and decreases by exactly one at each non-optimal step, within at most $M(\sigma, \sigma^*)$ steps the sequence must reach case (a): either a globally optimal $\lambda_k \neq \sigma^*$, or $M(\lambda_k, \sigma^*) = 0$ implying $\lambda_k = \sigma^*$. In either case a globally optimal order is reached by a finite sequence of moves in H . □

5. Neighbourhood Variants for IJSP

I instantiate five concrete neighbourhood variants within the $H(\sigma)$ framework. They differ in which subset of extreme critical arcs each one considers. Within a critical block $B = (o_1, \dots, o_k)$, an arc (o_i, o_{i+1}) is a *boundary arc* if $i = 1$ or $i = k - 1$, and an *interior arc* otherwise. All five variants inherit the three structural guarantees of Section 4 (feasibility, no-loss of improving

Critical block $B = (o_1, o_2, o_3, o_4, o_5)$ on machine M_k



	a_{12}	a_{23}	a_{34}	a_{45}
N_1	•	•	•	•
N_2	•	—	—	•
N_{ext}	•	$?_v$	$?_v$	•
N_8	•	—	—	• [‡]
N_3	•	•	•	• [†]

• always included — always excluded $?_v$ conditional on `isViableSwap` (N_{ext})

[†] N_3 additionally considers all $\binom{k}{2}$ contiguous subsequence reversals within each block of length k .

[‡] N_8 additionally generates extra-block reinsertion moves for each block endpoint (see Section 5.5).

Figure 2: Critical block $B = (o_1, \dots, o_5)$ on machine M_k and arcs considered by each neighbourhood variant.

neighbours, and connectivity) for their arc-swap component: N_2 and N_{ext} because they only exclude interior block arcs, which by Proposition 2 can never carry improving moves; N_3 because it is a strict superset of N_1 ; and N_8 because it contains all N_2 arc swaps (no-loss and connectivity) and its arc swaps are covered by Theorem 1 (feasibility). The fifth variant, N_8 , additionally extends N_2 with *extra-block reinsertion* moves that relocate critical block endpoints to positions outside the block on the same machine; these reinsertion moves are outside the scope of Section 4 and their feasibility is verified at runtime. N_8 is included as an exploratory variant whose empirical behaviour is analysed in Section 6. Figure 2 illustrates a critical block and the arcs included by each variant.

5.1. N_1 : All Extreme Critical Arcs

N_1 includes every disjunctive arc (x, y) such that x immediately precedes y on the same machine in $G(\sigma)$ and the arc lies on a critical path of $G^-(\sigma)$ or $G^+(\sigma)$. No positional filter is applied within the block: both boundary

and interior arcs of every extreme critical block are candidates. Generation cost: $O(|E|)$ per extreme graph.

5.2. N_2 : Block-Boundary Filter

N_2 restricts N_1 to arcs at the *boundary* of each extreme critical block. A block $B = \{o_1, \dots, o_k\}$ is a maximal contiguous subsequence of operations on the same machine that all lie on a single extreme critical path. N_2 retains the two boundary arcs (o_1, o_2) and (o_{k-1}, o_k) and discards interior arcs (o_i, o_{i+1}) , $2 \leq i \leq k-2$. This is the direct adaptation of [2] to the interval setting: by Proposition 2 applied simultaneously to $G^-(\sigma)$ and $G^+(\sigma)$, the discarded interior arcs cannot carry improving moves under any of the four rankings, so N_2 inherits the same theoretical guarantees as N_1 .

5.3. N_3 : Subsequence Reversals

N_3 extends the candidate set with reversals of *contiguous subsequences* within a critical block. For block $B = \{o_1, \dots, o_k\}$, in addition to the single-arc swaps of N_1 , N_3 considers every contiguous pair (o_i, o_j) with $j > i+1$ as a multi-step reversal candidate. This generates a strictly larger neighbourhood than N_1 .

5.4. N_{ext} : Extended Nowicki-Smutnicki Neighbourhood

N_{ext} adapts the extended neighbourhood of [3]. It starts from N_2 (boundary arcs unconditionally) and additionally considers interior block arcs that pass a heads-and-tails viability check, denoted `isViableSwap`:

- Boundary arcs (o_1, o_2) and (o_{k-1}, o_k) are always included (as in N_2).
- Interior arcs (o_i, o_{i+1}) , $2 \leq i \leq k-2$, are included only if `isViableSwap` (o_i, o_{i+1}) returns true.

`isViableSwap` (x, y) computes an optimistic lower-bound estimate of the makespan after swapping x and y by recalculating only the local heads and tails of x and y , without propagating changes through the full graph. Because the estimate is a lower bound, discarding a swap when the estimate exceeds the incumbent makespan is theoretically safe.

5.5. N_8 : Block-Boundary Swaps with Extra-Block Reinsertion

N_8 is an exploratory variant that extends N_2 with *extra-block reinsertion* moves, adapting the neighbourhood of [11] to the interval setting. For a critical block $B = (o_1, \dots, o_k)$ on a machine, with machine predecessor o_0 (if any) and machine successor o_{k+1} (if any), N_8 includes all N_2 boundary arc swaps and additionally considers four reinsertion candidates per block:

- move o_1 to just after o_k (between o_k and o_{k+1});
- move o_k to just before o_1 (between o_0 and o_1);
- move o_1 to the head of the machine (before all tasks on that machine);
- move o_k to the tail of the machine (after all tasks on that machine).

Each reinsertion candidate is evaluated by updating heads and tails via a localised BFS propagation; it is retained only if the resulting makespan mid-point improves on the current incumbent.

Because N_8 contains all N_2 boundary arc swaps, it satisfies all three structural guarantees of Section 4 for its arc-swap component: feasibility (Theorem 1), no-loss (Proposition 2, same argument as N_2), and connectivity (Theorem 2, since $N_8 \supseteq N_2$). The reinsertion moves are outside the scope of Section 4; their feasibility is verified at runtime and they carry no no-loss guarantee of their own. The motivation is the empirical question of whether extra-block moves benefit interval scheduling. As shown in Sections 6.5 and 6.6, they do not: N_8 is competitive on CRISP but underperforms in the genuinely interval-valued case, and Section 7.4 explains why.

6. Experimental Study

The experimental study is organised in three phases that address distinct questions.

Phase A (Sections 6.2 to 6.4). At a *common* hyperparameter setting, I compare the four ranking operators on five neighbourhoods (5×4 design), focusing on solution quality, the width of the returned interval, and the structural mechanism behind the LEX2-width effect.

Phase B (Section 6.5). After an irace-based tuning of the tabu search local search separately for each of the five neighbourhoods, I compare the five neighbourhoods on equal footing. The tuning isolates the structural effect of the neighbourhood from confounding parameter choices.

Phase C (Section 6.6). I validate Phase B on the 82 corresponding deterministic instances (CRISP), measuring the relative percentage deviation (RPD) from the best known solutions and comparing the relative ranking of the five neighbourhoods.

6.1. Setup Common to All Phases

Instances.. The benchmark suite contains 82 instances split into two groups. The first group (*classical benchmarks*) comprises 12 instances selected from the OR-Library [24]: abz7, abz8, abz9 (20×15); ft10 (10×10), ft20 (20×5); la21, la24, la25 (15×10), la27, la29 (20×10), la38, la40 (15×15). The second group comprises 70 instances from the Taillard benchmark [25], covering seven size classes with 10 instances each: 15×15 (225 operations), 20×15 (300), 20×20 (400), 30×15 (450), 30×20 (600), 50×15 (750) and 50×20 (1 000). All 82 instances are derived from their deterministic counterparts by perturbing each nominal processing time p_{ij}^* with a $\pm 7.5\%$ uniform symmetric interval, $\mathbf{p}_{ij} = [0.925 p_{ij}^*, 1.075 p_{ij}^*]$ (15% relative width).

Hardware and software.. All experiments were executed on an Intel Xeon E5-2680 v4 workstation (14 cores / 28 threads, 2.40 GHz, 16 GB RAM) running Windows 10 with a WSL2 Linux environment. The fEABC algorithm was implemented in C++14 and compiled with GCC using optimisation flags `-O3 -march=native -mtune=native -ffast-math`. To reduce wall-clock time, up to 24 independent runs were dispatched in parallel (one process per thread). Hyperparameter tuning (Phase B) used the irace package [26] (R 4.5.3), with a budget of 1 500 candidate evaluations per neighbourhood and a Friedman-test racing criterion; the tuning was parallelised over 24 cores and required approximately five days of wall-clock time across the five neighbourhoods. The subsequent Phase B/C evaluation (12 300 runs, 15-minute time limit per run) required approximately four additional days. Statistical analysis was performed in R with paired Wilcoxon tests and Holm–Bonferroni correction.

Algorithm.. All configurations use *fEABC* [27], a population-based hybrid metaheuristic that combines an Artificial Bee Colony (ABC) framework, Particle Swarm Optimisation (PSO) velocity updates for solution generation, and an embedded local search applied at each generation to a fraction of the population, with Lamarckian propagation of improvements. Solutions are encoded as permutations with repetition (*job-order*) and decoded via an insertion-based Schedule Generation Scheme (SGS); the population contains 250 individuals. The local search component differs by phase: Phase A uses first-improvement hill climbing as a neutral, operator-agnostic baseline; Phases B and C use a tabu search (best-neighbour selection with a short-term tabu list) independently tuned by irace for each neighbourhood (Section 6.5; the rationale for this switch is discussed there). The stopping criterion differs by phase: Phase A terminates after 25 consecutive generations without improvement in the global best; Phases B and C terminate after 20 such generations or after a wall-clock limit of 900 s (15 minutes) per run, whichever is reached first. Thirty independent runs are executed for every (instance, configuration) pair.

Hyperparameter configuration.. Phase A adopts the configuration validated in the *fEABC* paper [27]: JOX (Job Order Crossover), inversion mutation, food-source abandonment limit $NT_{\max} = 20$ (maximum trials before a source is abandoned), elite size 40, and first-improvement hill-climbing local search applied to 75% of individuals per generation.

Phases B and C replace hill climbing with tabu search and tune six parameters independently per neighbourhood using irace [26]: crossover operator, mutation operator, elite size, $NT_{\max}$, local-search target fraction, and the bad-iteration limit (maximum consecutive non-improving tabu iterations). The remaining parameters are fixed: population 250, tabu-list minimum size 3, and LEX2 ranking. LEX2 is chosen because Phase A shows it produces makespan intervals 5–11% narrower than the alternatives (Table 4) with no significant difference in midpoint quality versus EV ($p = 0.197$, $r = 0.026$, Table 3); fixing it allows the neighbourhood comparison in Phase B to be conducted under the operator that minimises solution uncertainty at no cost in makespan quality. Table 1 reports the resulting configurations.

Statistics.. Comparisons use the Friedman test (non-parametric repeated-measures ANOVA; the statistic follows a χ^2 distribution with $k - 1$ degrees of freedom for k groups) followed by pairwise Wilcoxon signed-rank tests with

Table 1: Irace-tuned hyperparameter configurations for Phases B and C. Parameters not listed are fixed: population 250, tabu-min-size 3, LEX2 ranking. *LS target*: fraction of individuals receiving local search per generation. *Bad-iter*: maximum consecutive non-improving tabu iterations before the local search terminates.

	Crossover	Mutation	Elite	NTmax	LS target	Bad-iter
N_1	JOX	insertion	33	11	85.3%	16
N_2	JOX	insertion	50	24	100%	20
N_3	GOX	swap	48	25	82.8%	30
N_{ext}	JOX	swap	39	17	76.2%	23
N_8	JOX	swap	39	27	100%	21

Table 2: Phase A: mean midpoint (mid.) and median wall-clock time t (s) per run, by neighbourhood and ranking operator (82 instances $\times$ 30 runs). For midpoint, lower is better.

Neighbourhood	EV		LEX1		LEX2		YX	
	mid.	t	mid.	t	mid.	t	mid.	t
N_1	1888.9	52	1894.6	33	1889.5	37	1887.0	32
N_2	1888.7	29	1895.2	41	1889.2	33	1887.1	29
N_3	1907.3	194	1914.2	206	1908.5	254	1904.9	201
N_8	1890.5	30	1894.6	33	1891.3	36	1887.5	31
N_{ext}	1889.0	33	1891.0	85	1895.5	65	1887.2	33

Holm–Bonferroni correction. Effect size is reported as $r = |Z|/\sqrt{N}$, where Z is the standardised Wilcoxon test statistic and N is the number of paired observations, with the conventional thresholds (small ≥ 0.1 , medium ≥ 0.3 , large ≥ 0.5). Each (instance, run) pair forms a paired block. The metric is the midpoint of the makespan interval, $(C_{\text{max}}^- + C_{\text{max}}^+)/2$, which is a neutral common ground for all four operators.

6.2. Phase A: Ranking Operator Comparison

The four operators are compared on five neighbourhoods (N_1 , N_2 , N_3 , N_8 , N_{ext}), giving $5 \times 4 = 20$ configurations and $20 \times 82 \times 30 = 49,200$ runs in total. Table 2 reports the global mean midpoint over all 82 instances. The Friedman test across operators yields $\chi^2(3) = 694.96$, $p < 10^{-150}$.

Pairwise tests on operators (Table 3) show that LEX1 is significantly worse than the other three (medium-to-large effects), while EV and LEX2 are statistically equivalent ($p = 0.197$, $r = 0.026$). YX is the best on average, but its advantage over EV and LEX2 carries only small effects.

Table 3: Phase A: pairwise Wilcoxon tests for ranking operators ($n = 2460$ paired blocks, Holm–Bonferroni corrected).

A	B	Mean A	Mean B	Diff	p_{adj}	r	Magnitude
LEX1	YX	1909.5	1901.5	+8.0	$< 10^{-3}$	0.509	large
EV	LEX1	1903.3	1909.5	−6.2	$< 10^{-3}$	0.415	medium
LEX1	LEX2	1909.5	1903.9	+5.6	$< 10^{-3}$	0.349	medium
LEX2	YX	1903.9	1901.5	+2.4	$< 10^{-3}$	0.159	small
EV	YX	1903.3	1901.5	+1.8	$< 10^{-3}$	0.131	small
EV	LEX2	1903.3	1903.9	−0.6	0.197	0.026	negligible

Runtime. The choice of ranking operator has no meaningful effect on wall-clock time. Kruskal–Wallis tests within each neighbourhood return $p > 0.48$ for N_1 , N_2 , N_3 , and N_8 ; the only significant result is on N_{ext} ($p = 0.005$), where LEX1 and LEX2 take roughly twice as long as EV or YX at the median, but all four remain fast in absolute terms (≤ 85 s). As Table 2 shows, the per-neighbourhood median runtimes differ by at most 60 s across operators within the same neighbourhood (N3: 194–254 s). The dominant driver of runtime is the neighbourhood structure, not the operator (N3 is $5\text{--}8\times$ slower than N2 at the median regardless of operator).

6.3. Phase A: Interval Width by Operator

The *width* of the returned makespan interval, $C_{\text{max}}^+ - C_{\text{max}}^-$, is a measure of solution *uncertainty* [4, 5]: the narrower the interval, the more precisely the planner can predict the actual makespan, regardless of which scenario realises. Figure 3(a) visualises the distribution of width by operator across all five neighbourhoods; the operator effect is consistent: LEX2 produces the narrowest intervals.

Table 4 quantifies the effect: LEX2 yields intervals $\sim 5\%$ narrower than EV and YX and $\sim 11\%$ narrower than LEX1. The ordering $\text{LEX1} \succ \text{EV} \succ \text{YX} \succ \text{LEX2}$ (in width, from widest to narrowest) holds in every neighbourhood; the choice of neighbourhood has a negligible effect on width (< 3 units within any operator).

The advantage of LEX2 is not merely a difference in means: Figure 4 plots the width returned by LEX2 against each of the three alternatives, one point per (instance, neighbourhood) pair. In all three comparisons the majority of points fall below the diagonal — 89% of cases against LEX1, and around 71% against EV and YX — confirming that the effect is consistent

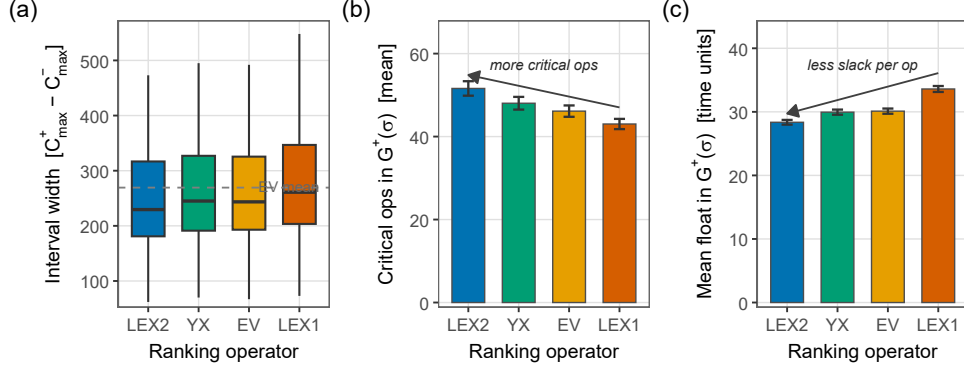


Figure 3: Phase A: operator comparison (5 neighbourhoods $\times$ 82 instances = 410 observations per operator; one best-of-30 solution per configuration). (a) Distribution of makespan interval width $C_{\max}^+ - C_{\max}^-$; the dashed line marks the EV mean. (b) Mean number of critical operations in $G^+(\sigma)$ (i.e., operations with $F^+ = 0$). (c) Mean per-operation float in $G^+(\sigma)$. Panels (b) and (c) point in the same direction: LEX2 has *more* critical operations and *less* per-operation slack in $G^+(\sigma)$, confirming that LEX2 tightens the worst-case critical path directly rather than creating more structural slack.

Table 4: Mean makespan interval width by ranking operator (Phase A; mean over 5 neighbourhoods $\times$ 82 instances; one best-of-30 solution per configuration).

Operator	Mean width	vs. EV
EV	269.3	—
LEX1	286.4	+6.4%
LEX2	255.0	−5.3%
YX	267.5	−0.7%

across instances and neighbourhoods, though less dominant against the two mid-range operators.

6.4. Phase A: Why does LEX2 Narrow Intervals? A Float Analysis

The natural hypothesis (H2) for the LEX2-width effect is that LEX2 leaves *more structural slack* in $G^+(\sigma)$, which would let the worst-case path shorten. The alternative hypothesis (H1) is that LEX2 *tightens* $G^+(\sigma)$ directly by accepting moves that reduce $C_{\max}^+$ even when $C_{\max}^-$ does not improve. To discriminate, I reconstruct $G^-(\sigma)$ and $G^+(\sigma)$ from the saved best-of-30 permutation for every (configuration, instance) pair and compute the float of every operation in both graphs.

Figures 3(b) and 3(c) report two derived statistics: the mean number of

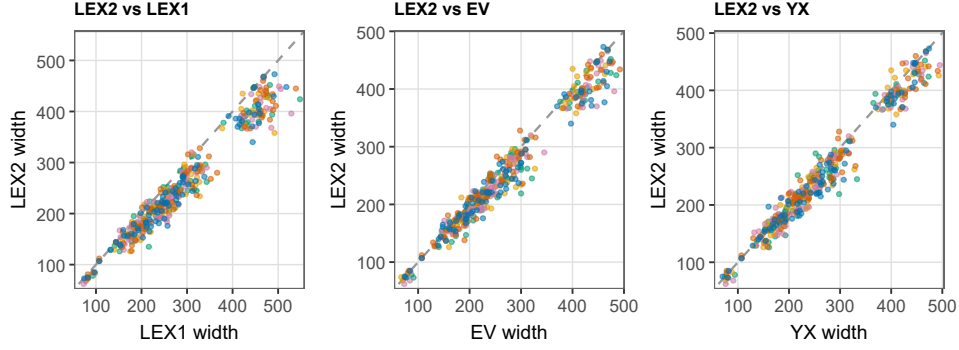


Figure 4: Paired comparison of makespan interval width: LEX2 versus each alternative operator (Phase A; 5 neighbourhoods $\times$ 82 instances = 410 points per panel, one best-of-30 solution per configuration; colour encodes neighbourhood). Points below the dashed diagonal indicate cases where LEX2 produces a narrower interval: 88.8% of cases vs. LEX1, 72.2% vs. EV, and 70.5% vs. YX.

critical operations in $G^+(\sigma)$ (i.e., operations with $F^+ = 0$) and the mean per-operation float in $G^+(\sigma)$, respectively. Both panels point in the *same* direction: LEX2 has *more* critical operations in $G^+(\sigma)$ and *less* per-operation slack. This is the opposite of what H2 predicts and exactly what H1 predicts.

The asymmetry between the two extreme graphs is the structural fingerprint of LEX2: the operator pushes optimisation effort entirely toward the upper-bound graph $G^+(\sigma)$ (where it has more critical ops and less slack), at the cost of a relatively looser $G^-(\sigma)$ (fewer critical ops there). Both effects are mediated by the ranking, not by a coincidental property of the search trajectory.

6.5. Phase B: Neighbourhood Comparison after Tuning

Phase A used a common hyperparameter setting that prevents over-fitting to a single neighbourhood, but it leaves open the possibility that any of the five neighbourhoods would benefit from its own tuned configuration. The transition from hill climbing to tabu search also reflects the change in experimental objective: hill climbing converges quickly to a local optimum regardless of which neighbourhood is used, so once the search stalls the structural differences between neighbourhoods are no longer visible; tabu search continues to move past local optima, which means each neighbourhood’s coverage, theoretical guarantees, and block geometry can influence the search trajectory throughout the full time budget—the necessary condition for a mean-

Table 5: Phase B: pairwise Wilcoxon tests for tuned neighbourhoods ($n = 2460$ paired blocks, Holm–Bonferroni corrected). All differences are statistically significant; the magnitude of the effect indicates the practical relevance.

A	B	Mean A	Mean B	Diff	p_{adj}	r	Magnitude
N_3	N_{ext}	1898.8	1882.1	+16.7	$< 10^{-3}$	0.530	large
N_1	N_3	1884.0	1898.8	−14.7	$< 10^{-3}$	0.469	medium
N_{ext}	N_8	1882.1	1896.9	−14.9	$< 10^{-3}$	0.458	medium
N_1	N_8	1884.0	1896.9	−12.9	$< 10^{-3}$	0.407	medium
N_2	N_3	1887.6	1898.8	−11.2	$< 10^{-3}$	0.385	medium
N_2	N_8	1887.6	1896.9	−9.4	$< 10^{-3}$	0.320	medium
N_2	N_{ext}	1887.6	1882.1	+5.5	$< 10^{-3}$	0.171	small
N_1	N_{ext}	1884.0	1882.1	+2.0	$< 10^{-3}$	0.097	negligible
N_1	N_2	1884.0	1887.6	−3.5	$< 10^{-3}$	0.076	negligible
N_3	N_8	1898.8	1896.9	+1.8	$< 10^{-3}$	0.067	negligible

ingful neighbourhood comparison. Using the irace-tuned configurations in Table 1, I evaluate the five neighbourhoods on the 82 benchmark instances (30 runs per instance, $5 \times 82 \times 30 = 12,300$ observations); the most notable configuration difference is that irace selected GOX crossover for N_3 —the only variant using subsequence reversals—while all others retained JOX, and N_3 received the highest bad-iteration budget (30), consistent with its larger candidate set.

Figure 5(a) shows the per-instance RPD (relative percentage deviation):

$$\text{RPD}(\sigma) = \frac{C_{\text{max}}(\sigma) - C_{\text{ref}}}{C_{\text{ref}}} \times 100, \quad (12)$$

where C_{ref} is a phase-dependent reference: in Phase B, where no best known solution exists for the interval problem, C_{ref} is the per-instance minimum midpoint achieved across all five tuned neighbourhoods on the same run; in Phase C (CRISP validation, Section 6.6), C_{ref} is the best known solution (BKS) from the literature [25, 24, 28]. Values of zero in Phase B correspond to the per-instance winner, and higher values measure relative degradation. The Friedman test yields $\chi^2(4) = 1113.03$, $p < 10^{-150}$; mean ranks are N_{ext} : 2.465, N_1 : 2.667, N_2 : 2.739, N_8 : 3.479, N_3 : 3.651. Pairwise tests (Table 5) confirm a clean partition into a top group $\{N_{\text{ext}}, N_1, N_2\}$, with N_8 and N_3 trailing by a medium-effect margin.

The picture that emerges is sharper than under the common hyperpa-

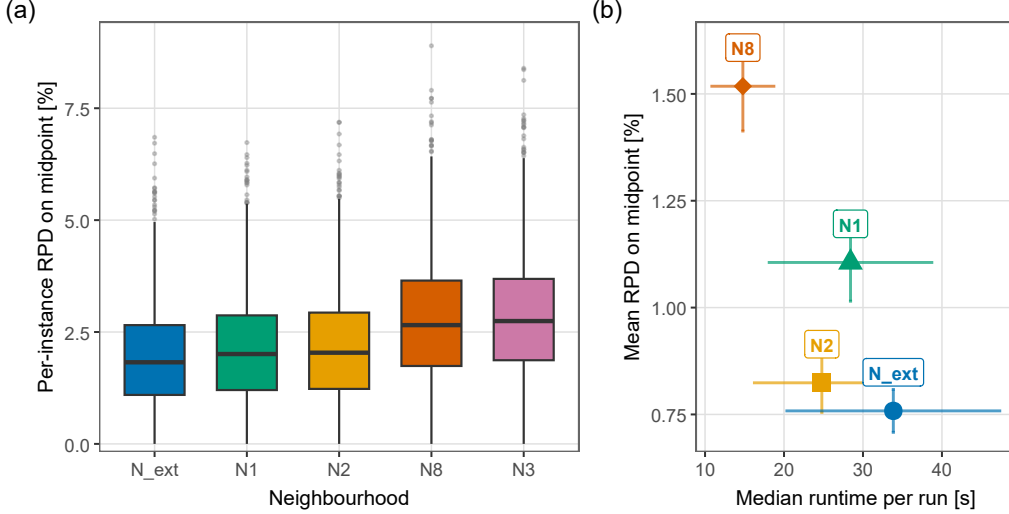


Figure 5: Phase B (irace-tuned hill climbing). **(a)** Per-instance RPD on the makespan midpoint by neighbourhood. Each box pools $82 \times 30 = 2,460$ observations; the per-instance baseline is the minimum midpoint across all five tuned neighbourhoods on the same run. Pairwise significance and effect sizes are reported in Table 5. **(b)** Quality–runtime trade-off (N3 omitted for scale clarity): each point is the mean RPD vs. mean median runtime aggregated over all instance groups; error bars are ± 1 SE. N_{ext} achieves the lowest RPD at moderate runtime (≈ 34 s); N_8 is the fastest (≈ 15 s) but trails on quality by a medium effect.

parameter setting of Phase A: N_{ext} overtakes N_1 and N_2 as the strongest neighbourhood after tuning (small but consistent effect), and the new N_8 variant is a medium-effect step *worse* than the top group. Figure 5(b) aggregates this quality–runtime trade-off visually: N_{ext} achieves the lowest mean RPD at moderate runtime, while N_8 is the fastest but pays the medium-effect quality penalty. The latter is, at first sight, surprising: N_8 adds extra-block reinsertion moves on top of N_2 , which seemed a natural extension for escaping local optima. Section 6.6 shows that the reinsertions *do* help when only one graph exists (CRISP), but in the interval setting their midpoint-based filter discards moves that reduce C_{max}^+ independently of C_{max}^- — precisely those that exploit the disagreement between $G^-(\sigma)$ and $G^+(\sigma)$.

Table 6 decomposes the results by instance group and simultaneously reports median runtime t (seconds), bringing the cost side into view. N_{ext} posts the lowest group-level RPD on 9 of 19 groups; N_2 leads on 8 (all Fisher–Thompson, ABZ, and most Lawrence instances); N_1 leads only on

the two largest Taillard classes, where its larger swap-based neighbourhood may benefit from the extended time budget. The runtime columns reveal a clear cost–quality trade-off: N_8 is the fastest neighbourhood throughout—running at roughly half the wall-clock time of N_{ext} on the largest instances (tai50 $\times$)—yet trails on solution quality by a medium effect. N_3 occupies the worst position on both dimensions: at tai50 $\times$ 20 its median runtime of 725s is 10 $\times$ that of N_8 and 3 $\times$ that of N_{ext} , with no compensating quality gain on any group.

Considering quality and time jointly sharpens the comparison between N_{ext} and N_2 . Across all 82 instances, both have essentially the same global median runtime (22s vs. 21s), while N_{ext} leads on mean RPD by only 0.23 percentage points (1.92% vs. 2.15%). However, this balance shifts on the large Taillard classes: at tai50 $\times$ 15 and tai50 $\times$ 20, N_2 ’s median runtimes (91s and 154s) are roughly one-third lower than N_{ext} ’s (160s and 229s), while the quality gap narrows to at most 0.48 percentage points. The practical implication is that N_2 offers the best efficiency profile for large instances—delivering near-top quality at substantially lower cost—whereas N_{ext} is preferable when solution quality is the sole criterion, regardless of instance size.

6.5.1. Convergence

Figure 6 compares the per-generation convergence profiles of the five tuned neighbourhoods, normalised so that every trace starts at 1.000 at generation 0. The top group (N_{ext}, N_1, N_2) is essentially indistinguishable in convergence speed, all reaching their plateau by generation ~ 50 . N_3 converges slightly slower and to a worse plateau. N_8 exhibits a clear gap: it converges noticeably more slowly and its normalised plateau is approximately 1.04 against ~ 1.00 for the top group, i.e., it stalls at roughly 4% higher relative makespan. This is consistent with its higher per-iteration cost: each reinsertion candidate requires a BFS head-propagation pass, and many such candidates do not improve the interval criterion, leading to slower convergence at lower yield.

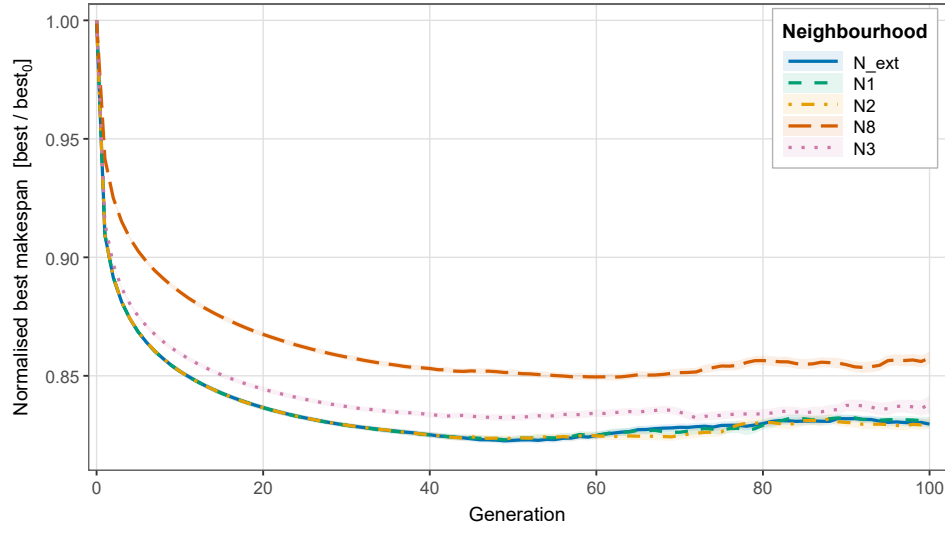


Figure 6: Per-generation normalised best-makespan profiles, by neighbourhood (Phase B; mean over 82 instances). The shaded ribbons represent ± 1 standard error of the mean. The top group (N_{ext}, N_1, N_2) overlaps tightly; N_8 converges noticeably slower than the top group; N_3 plateaus at a worse value.

Table 6: Phase B: mean RPD (%) relative to the per-instance best-found midpoint across all five tuned configurations, and median runtime t (s), by instance group and neighbourhood. Bold marks the per-row minimum RPD. Taillard rows aggregate the 10 instances of each class.

	N_{ext}		N_1		N_2		N_8		N_3	
	%	t	%	t	%	t	%	t	%	t
<i>Fisher & Thompson</i>										
ft10 (10×10)	1.17	3	2.17	4	0.80	3	2.36	3	2.10	7
ft20 (20×5)	1.08	5	1.14	5	1.02	4	1.17	4	1.54	8
<i>Adams–Balas–Zawack</i>										
abz7 (20×15)	1.81	13	2.18	14	2.09	12	2.26	9	2.20	45
abz8 (20×15)	1.66	15	2.04	14	1.59	13	2.40	10	2.61	48
abz9 (20×15)	3.19	14	3.43	14	2.94	14	4.35	10	5.00	44
<i>Lawrence</i>										
la21 (15×10)	1.10	6	2.17	7	1.16	5	2.21	5	2.71	12
la24 (15×10)	1.53	5	1.90	5	1.45	5	1.83	5	2.19	12
la25 (15×10)	1.73	5	1.98	6	1.53	5	2.01	5	2.43	11
la27 (20×10)	0.99	10	1.47	9	1.23	8	1.64	7	1.92	19
la29 (20×10)	2.25	12	3.13	12	1.95	10	3.88	8	4.30	22
la38 (15×15)	1.58	7	2.67	8	2.16	7	2.66	6	3.20	22
la40 (15×15)	1.64	7	1.25	8	1.14	7	1.74	6	1.59	24
<i>Taillard (mean RPD / median t over 10 instances)</i>										
15×15	1.84	8	2.03	8	1.86	8	2.52	6	2.56	22
20×15	2.09	17	2.47	15	2.24	15	3.10	10	3.11	49
20×20	2.07	21	2.29	19	2.27	20	2.90	13	2.78	84
30×15	1.81	45	1.97	39	2.13	38	2.75	22	2.88	141
30×20	2.40	61	2.60	51	2.70	52	3.39	30	3.50	267
50×15	1.29	160	1.28	124	1.77	91	2.18	50	2.33	358
50×20	2.28	229	2.01	178	2.79	154	2.78	72	2.93	725

6.6. Phase C: Validation on Deterministic Instances (CRISP)

To check whether the relative ranking of neighbourhoods carries over to the deterministic setting, I apply the same five tuned configurations of Phase B to the 82 *crisp* (deterministic, CRISP) counterparts of the benchmark instances, obtained by setting $\mathbf{p}_{ij} = [p_{ij}^*, p_{ij}^*]$ (equivalently $G^-(\sigma) = G^+(\sigma)$). The metric is RPD (Equation 12) with $C_{\text{ref}} = \text{BKS}$, as defined in Section 6.5.

Table 7 reports global summary statistics. The relative ordering of the five neighbourhoods on CRISP is $N_{\text{ext}} \succeq N_8 \succeq N_1 \succeq N_2 \succeq N_3$, with N_{ext} leading on mean RPD (3.18%). N_8 is genuinely competitive on CRISP — a marked contrast with its Phase B performance.

Table 7: Phase C: CRISP validation. RPD against best known solutions over the 82 instances and 30 runs.

N	Mean RPD (%)	Median RPD (%)	SD	Max (%)	Median t (s)
N_{ext}	3.175	2.988	2.195	9.101	25
N_8	3.290	2.960	2.372	9.751	25
N_1	3.359	3.219	2.336	9.158	20
N_2	3.374	3.197	2.307	9.751	20
N_3	3.802	3.729	2.523	10.130	36

The CRISP validation supports the interpretation of N_8 ’s behaviour: extra-block reinsertions provide a genuine structural advantage when a single critical graph exists (deterministic), but in the interval setting the midpoint-based reinsertion filter discards moves that reduce C_{max}^+ independently of C_{max}^- — precisely those that exploit the disagreement between $G^-(\sigma)$ and $G^+(\sigma)$. This is, to the best of my knowledge, the first experimental evidence that a CRISP-good neighbourhood can be IJSP-bad for purely structural reasons.

7. Discussion

7.1. Practical Recommendations

The experimental results lead to a small set of actionable recommendations:

1. Choose the neighbourhood based on instance size. For small and medium instances (up to approximately 30×20), N_{ext} and N_2 have essentially the same median runtime (22 s vs. 21 s globally), so N_{ext} is

the preferred choice: it leads on quality with a small but consistent effect over N_2 ($r = 0.171$) and a negligible gap over N_1 ($r = 0.097$), which is itself a strong alternative. For large instances (tai50×15 and tai50×20), N_2 is approximately one-third faster than N_{ext} (91 s vs. 160 s and 154 s vs. 229 s) while the quality penalty remains small, making N_2 the most efficient choice when throughput matters.

2. Use LEX2 if interval width is the goal; operator choice has limited impact on midpoint. LEX2 yields makespan intervals 5–11% narrower than the alternatives on average, with structural evidence for this effect (Section 7.2), and this advantage is consistent across prior work that includes per-operator hyperparameter tuning [29, 30]. For the midpoint objective, YX shows a slight advantage over EV at the common hyperparameter setting of Phase A, but prior studies with independent per-operator tuning on 12 small benchmark instances find that differences between operators largely disappear once each is individually configured [29, 30]; whether this holds for larger instances remains an open question.
3. Avoid N_8 for genuinely interval-valued problems. Its midpoint-based reinsertion filter is informative on CRISP but counter-productive on intervals: it discards moves that reduce C_{max}^+ independently of C_{max}^- , the very moves that exploit interval-specific structure.
4. Avoid N_3 in either setting. The multi-step subsequence reversals it adds beyond N_1 are dominated on both dimensions: it is the weakest neighbourhood on solution quality and the most expensive computationally (95 s global median, against 13–22 s for all other variants).

7.2. Theoretical Connection: Why LEX2 Narrows Intervals

Proposition 2 guarantees that reversing a non-extreme-critical arc cannot improve either bound. LEX2 specifically targets C_{max}^+ , accepting moves that reduce C_{max}^+ even when C_{max}^- stays put. Over many local search steps, this creates a bias toward configurations where $G^+(\sigma)$ has a shorter critical path — and, by the structure of interval arithmetic, a tighter bound on the overall interval width. The float analysis of Section 6.4 provides direct empirical confirmation: LEX2 solutions have more critical operations in $G^+(\sigma)$ and less per-operation slack, while $G^-(\sigma)$ becomes relatively looser. The asymmetry between the two extreme graphs is a structural fingerprint of the LEX2 ranking rule.

7.3. Why N_3 Underperforms

N_3 is counter-intuitive: it is a strict superset of N_1 in the move-generation sense (single-arc boundary swaps plus three types of 3-element subsequence reversals), yet it produces worse solutions. Each multi-arc reversal candidate requires a full feasibility evaluation to detect cycles before it can be considered, which significantly increases the per-iteration cost. More importantly, these extra candidates do not carry improving moves in practice: the boundary arc swaps already capture the structurally meaningful transitions (as justified by Proposition 2), and the subsequence reversals add volume to the candidate pool without adding quality. Under best-improvement selection, a larger pool diluted with poor-quality candidates reduces the effective yield per evaluation, resulting in slower convergence to a worse plateau.

7.4. Why N_8 Is CRISP-Good but IJSP-Bad

N_8 extends N_2 with extra-block reinsertion moves filtered by a midpoint lower-bound estimate. In the deterministic setting this pairing is productive: with a single critical path, a reinsertion that improves the midpoint also improves the makespan, and the extra moves frequently find improvements that the boundary arc swaps alone would miss [11]. In the interval setting, the filter loses this calibration. Under LEX2, the most valuable moves are precisely those that reduce $C_{\max}^+$ even when $C_{\max}^-$ does not improve; such moves have a *worse* midpoint and are therefore discarded. Moreover, a reinsertion position that shortens the critical path in $G^-(\sigma)$ cannot generally shorten the path in $G^+(\sigma)$ simultaneously, because the two extreme graphs can disagree on which operations are critical. The net effect is that N_8 spends computational budget evaluating reinsertion moves that rarely improve the interval criterion — each requiring a BFS head-propagation pass — while N_2 focuses the same budget on boundary arc swaps with a proven link to the interval critical structure.

7.5. Limitations and Future Work

- Larger instances and per-operator tuning. Extending to $\text{tai}100 \times 20$ (2000 operations) would require excessively long runtimes on the current cluster, but would be valuable: larger search spaces tend to sharpen structural differences between neighbourhoods, and the question of whether operator differences persist under per-operator tuning remains open beyond the 12 small instances studied in [29, 30].

- Other uncertainty levels. The 15% uniform interval width is a single noise level; sensitivity to other noise widths is left open.
- Hybrid algorithms. $(\text{LEX2}, N_{\text{ext}})$ is the natural starting point for an uncertainty-aware metaheuristic targeting interval width; $(\text{LEX2}, N_2)$ offers a faster alternative with a small quality trade-off (Section 7.1). A dedicated algorithm that exploits the structural mechanism of Section 7.2—directly optimising moves that tighten $G^+(\sigma)$ —could improve on the general-purpose tabu search used here.
- Other rankings. Additional admissible orders on intervals [6] could be tested with the same protocol; the structural mechanism uncovered for LEX2 should generalise to any operator that strictly prioritises one extreme.

8. Conclusions

I have presented a theoretical framework for neighbourhood-based local search in the Interval Job Shop Scheduling Problem grounded in the concept of *extreme critical paths* — arcs that are critical in either the best-case or worst-case scenario graph. The resulting neighbourhood $H(\sigma)$ satisfies feasibility, connectivity and the no-loss-of-improving-neighbours property for all four admissible interval rankings considered (EV, LEX1, LEX2, YX) simultaneously.

Five neighbourhood variants ($N_1, N_2, N_3, N_{\text{ext}}, N_8$) and four ranking operators were evaluated on 82 benchmark instances of up to 1000 operations, in three experimental phases: a 5×4 comparison at a common parameter setting, a five-way comparison after irace-based tuning, and a CRISP validation on the corresponding deterministic instances.

After per-neighbourhood tuning, N_{ext} is the strongest variant, followed by N_1 and N_2 within a small effect; N_8 and N_3 trail by a medium effect. Considered jointly with runtime, N_2 emerges as the most efficient choice for large instances, delivering near-top quality at roughly one-third lower cost than N_{ext} . Regarding ranking operators, LEX2 yields makespan intervals 5–11% narrower than the alternatives on average. A float analysis of both extreme graphs provides structural support for this finding: LEX2 solutions show more critical operations and less per-operation slack in $G^+(\sigma)$, consistent with LEX2 directly tightening the worst-case critical path rather than

leaving more structural slack. For the midpoint objective, differences between operators are small at a common parameter setting and prior work suggests they largely disappear with per-operator tuning. Finally, the N_8 variant (extra-block reinsertion on top of N_2 , after [11]) is competitive on CRISP but underperforms on intervals — a counter-intuitive finding traced back to the mismatch between its midpoint-based reinsertion filter and the interval criterion: moves that reduce $C_{\max}^+$ without improving the midpoint are discarded, costing the search the degrees of freedom that distinguish IJSP from JSP.

These results provide actionable guidance for practitioners implementing local search for uncertain job shop scheduling: the choice of ranking operator can be used not only to optimise expected makespan but also to directly control solution uncertainty, and the choice of neighbourhood should account for whether the problem is genuinely interval-valued or only nominally uncertain.

CRedit authorship contribution statement

Hernán Díaz Rodríguez: Conceptualization, Methodology, Software, Formal analysis, Investigation, Data curation, Visualization, Writing – original draft, Writing – review and editing.

Declaration of competing interest

The author declares that there are no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

[TODO: Confirm or replace.] The benchmark instances, source code of the local-search engine, irace tuning scripts, and the raw experimental data underlying every figure and table in this article are openly available at <https://github.com/<TODO-org>/<TODO-repo>>. The classical JSP benchmarks used to derive the interval instances are taken from the OR-Library and from [28].

Acknowledgements

[TODO: Funding sources, project IDs, computing resources.]

References

- [1] P. J. M. van Laarhoven, E. H. L. Aarts, J. K. Lenstra, Job shop scheduling by simulated annealing, *Operations Research* 40 (1) (1992) 113–125.
- [2] E. Nowicki, C. Smutnicki, A fast taboo search algorithm for the job shop problem, *Management Science* 42 (6) (1996) 797–813. doi:10.1287/mnsc.42.6.797.
- [3] E. Nowicki, C. Smutnicki, An advanced tabu search algorithm for the job shop problem, *Journal of Scheduling* 8 (2005) 145–159. doi:10.1007/s10951-005-6364-5.
- [4] R. E. Moore, R. B. Kearfott, M. J. Cloud, *Introduction to Interval Analysis*, Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 2009.
- [5] D. Lei, Interval job shop scheduling problems, *International Journal of Advanced Manufacturing Technology* 60 (2012) 291–301. doi:10.1007/s00170-011-3600-3.
- [6] H. Bustince, J. Fernandez, A. Kolesárová, R. Mesiar, Generation of linear orders for intervals by means of aggregation functions, *Fuzzy Sets and Systems* 220 (2013) 69–77. doi:10.1016/j.fss.2012.07.015.
- [7] S. Karmakar, A. K. Bhunia, A comparative study of different order relations of intervals, *Reliable Computing* 16 (2012) 38–72.
- [8] D. Dubois, H. Fargier, P. Fortemps, Fuzzy scheduling: Modelling flexible constraints vs. coping with incomplete knowledge, *European Journal of Operational Research* 147 (2003) 231–252.
- [9] C. Artigues, C. Briand, T. Garaix, Temporal analysis of projects under interval uncertainty, in: C. Schwindt, J. Zimmermann (Eds.), *Handbook on Project Management and Scheduling Vol. 2*, Springer, 2015, pp. 911–927. doi:10.1007/978-3-319-05915-0_11.
- [10] I. González Rodríguez, C. R. Vela, J. Puente, R. Varela, A new local search for the job shop problem with uncertain durations, in: *Proceedings of the Eighteenth International Conference on Automated Planning and Scheduling (ICAPS-2008)*, AAAI Press, Sydney, Australia, 2008, pp. 124–131.

- [11] J. Xie, X. Li, L. Gao, L. Gui, A new neighbourhood structure for job shop scheduling problems, *International Journal of Production Research* 61 (7) (2022) 2147–2161. doi:10.1080/00207543.2022.2053490.
- [12] Z. Xu, R. R. Yager, Some geometric aggregation operators based on intuitionistic fuzzy sets, *International Journal of General Systems* 35 (4) (2006) 417–433. doi:10.1080/03081070600574353.
- [13] I. González Rodríguez, C. R. Vela, J. Puente, A. Hernández-Arauzo, Improved local search for job shop scheduling with uncertain durations, in: *Proceedings of the Nineteenth International Conference on Automated Planning and Scheduling (ICAPS 2009)*, AAAI Press, Thessaloniki, Greece, 2009, pp. 154–161.
- [14] J. Puente, C. R. Vela, I. González Rodríguez, Fast local search for fuzzy job shop scheduling, in: H. Coelho, et al. (Eds.), *Proceedings of the 19th European Conference on Artificial Intelligence (ECAI 2010)*, IOS Press, 2010, pp. 739–744. doi:10.3233/978-1-60750-606-5-739.
- [15] C. R. Vela, S. Afsar, J. J. Palacios, I. González-Rodríguez, J. Puente, Evolutionary tabu search for flexible due-date satisfaction in fuzzy job shop scheduling, *Computers & Operations Research* (2020). doi:10.1016/j.cor.2020.104931.
- [16] H. Díaz, I. Díaz, I. González-Rodríguez, J. J. Palacios, C. R. Vela, A genetic approach to the job shop scheduling problem with interval uncertainty, in: *Information Processing and Management of Uncertainty in Knowledge-Based Systems (IPMU 2020)*, Vol. 1238 of *Communications in Computer and Information Science*, Springer, 2020, pp. 663–676. doi:10.1007/978-3-030-50143-3_52.
- [17] H. Díaz, J. J. Palacios, I. Díaz, C. R. Vela, I. González-Rodríguez, Tardiness minimisation for job shop scheduling with interval uncertainty, in: *Hybrid Artificial Intelligence Systems (HAIS 2020)*, Vol. 12344 of *Lecture Notes in Artificial Intelligence*, Springer, 2020, pp. 209–220. doi:10.1007/978-3-030-61705-9_18.
- [18] K. Tamssaouet, S. Dauzère-Pérès, A general efficient neighborhood structure framework for the job-shop and flexible job-shop scheduling problems, *European Journal of Operational Research* 311 (2) (2023) 455–471. doi:10.1016/j.ejor.2023.05.018.

- [19] P. García Gómez, S. Dauzère-Pérès, C. R. Vela, I. González-Rodríguez, New bounds for move evaluation in the flexible job-shop scheduling problem, *Computers & Industrial Engineering* 212 (2026) 111696. doi:10.1016/j.cie.2025.111696.
- [20] A. Calamita, C. Meloni, M. Pranzo, M. Samà, Fast risk estimation for job shop scheduling solutions under interval uncertainty via machine learning, *Flexible Services and Manufacturing Journal* (2025). doi:10.1007/s10696-025-09640-7.
- [21] J. Fortin, P. Zieliński, D. Dubois, H. Fargier, Criticality analysis of activity networks under interval uncertainty, *Journal of Scheduling* 13 (6) (2010) 609–627. doi:10.1007/s10951-010-0163-3.
- [22] D. Dubois, H. Fargier, J. Fortin, Computational methods for determining the latest starting times and floats of tasks in interval-valued activity networks, *Journal of Intelligent Manufacturing* 16 (2005) 407–421.
- [23] T. Garaix, C. Artigues, C. Briand, Fast minimum float computation in activity networks under interval uncertainty, *Journal of Scheduling* (2013). doi:10.1007/s10951-012-0272-2.
- [24] J. E. Beasley, OR-Library: distributing test problems by electronic mail, *Journal of the Operational Research Society* 41 (11) (1990) 1069–1072. doi:10.1057/jors.1990.166.
- [25] É. Taillard, Benchmarks for basic scheduling problems, *European Journal of Operational Research* 64 (2) (1993) 278–285. doi:10.1016/0377-2217(93)90182-M.
- [26] M. López-Ibáñez, J. Dubois-Lacoste, L. Pérez Cáceres, M. Birattari, T. Stützle, The irace package: Iterated racing for automatic algorithm configuration, *Operations Research Perspectives* 3 (2016) 43–58. doi:10.1016/j.orp.2016.09.002.
- [27] H. Diaz, J. J. Palacios, I. González-Rodríguez, C. R. Vela, Fast elitist ABC for makespan optimisation in interval JSP, *Natural Computing* 22 (2023) 645–657. doi:10.1007/s11047-023-09953-2.
- [28] Optimizier, Best known solutions for the Taillard job shop instances, <http://optimizier.com/TA.php>, accessed: May 2026.

- [29] H. Díaz, J. J. Palacios, I. González-Rodríguez, C. R. Vela, An elitist seasonal artificial bee colony algorithm for the interval job shop, *Integrated Computer-Aided Engineering* 30 (2023) 223–242. doi:10.3233/ICA-230705.
- [30] H. Díaz, J. J. Palacios, I. Díaz, C. R. Vela, I. González-Rodríguez, Robust schedules for tardiness optimisation in job shop with interval uncertainty, *Logic Journal of the IGPL* (2022). doi:10.1093/jigpal/jzac016.